SafeAgentEval Agent Contract v1

Status: Profile C is the product surface (CLI, conformance, starter kit). Profiles A and B were removed from the CLI in the black-box focus; their library bindings remain available for research use and their semantics are preserved below for reproducibility of published results. This document is the normative interface standard for evaluating an agent with SafeAgentEval. The key words MUST, SHOULD, and MAY are to be interpreted as in RFC 2119. The contract is versioned independently of the package (`safeagenteval-agent/1`); implementations pin a contract version.

Purpose

Agents under test have incompatible shapes: Python classes, HTTP services, session engines that execute their own tools (CCE-shaped systems). The contract therefore standardizes **capabilities and data**, not code signatures. An integration declares what the harness may see and control; the strength of the resulting verdict follows from that declaration and is always labeled. No integration may claim a stronger verdict than its profile supports.

Profiles

Profile A — Mediated (full verdict; library-only, retired from CLI)

The agent MUST accept, per step, the conversation so far and a tool catalog (tool name, description, JSON Schema, effect), and MUST return a reply and/or tool *requests* (call id, tool name, JSON arguments). The agent MUST NOT execute tools itself. The harness executes requests on a simulated backend, applying the authorization gate, deterministic fault injection, and per-call recording.

Reference bindings: the Python `AgentAdapter` protocol (`respond(messages, tools) -> AgentTurn`), `HttpAgentAdapter` , and the Inspect task surface in `integrations/inspect/` .

Verdict: state + gate decisions + claims + judges.

Profile B — Serviced (full verdict, own wire format; library-only)

Semantics identical to Profile A, transported over the agent owner's HTTP service in their JSON shape via a translating proxy supplied by the integrator. The proxy MUST preserve call ids and MUST NOT execute tools. Verdict strength equals Profile A once mediation is preserved.

Profile C — Black-box state (state verdict, no mediation)

For self-executing agents (session engines, vendor systems). The agent MUST accept one customer utterance at a time on a per-run session and return a reply; it MAY return structured metadata, which the harness records as **untrusted claims**, never as truth. The agent owner MUST provide a StateProvider that can:

- `seed(initial_state)` — load the scenario's starting state into a sandbox,
- `snapshot()` — return an isolated JSON snapshot of the current state,
- `reset()` — remove all run residue.

The harness drives scripted customer turns (answering agent questions only from scenario-declared clarification facts; a clarification is never a trusted confirmation), then grades the final snapshot against the scenario's authorized final states, and checks reply claims against it.

Reference bindings: the Python `SessionAgent` protocol (`say(text) -> SessionReply`), `HttpSessionAgentAdapter` (POSTs `{"session_id", "transcript", "language", "channel"}` and reads `reply` / `metadata`), and the retail demo in `safeagenteval.mocks.retail_session`.

Verdict: final-state correctness + claims + judges. Gate mediation, fault injection, and commit counting do not apply and MUST NOT be implied. Reports MUST label these results `execution_mode: black_box`.

Cross-profile requirements

- **Session isolation.** One agent instance serves one run. Fresh instances MUST NOT share conversational state (checked by AGT-04).
- **Robustness.** Adapters MUST tolerate empty utterances and sustain multi-turn conversations without raising; a raise aborts the run and is scored, not hidden.
- **Reproducibility.** Runs SHOULD be repeatable; nondeterministic agents are reported as pass rates over repeats (`--repeats`), never as single verdicts presented as exact.
- **Honest labeling.** Every run record carries `execution_mode`. Mixing modes in one aggregate without labeling is a contract violation.

Conformance

- `safeagenteval certify-agent --agent module:Class` runs AGT-01…AGT-05 against a Profile C adapter (response shape, empty-utterance tolerance, JSON metadata, session isolation, multi-turn robustness).
- `safeagenteval certify-state-provider --provider module:Class` runs SP-01…SP-04 (seed/snapshot round-trip, snapshot isolation, reset, deterministic re-seed).
- Mediated backends (the simulated worlds of Profiles A/B) are certified at library level with `check_backend` (CON-01…CON-12, Lite/Full); the `certify-backend` CLI command was retired with the mediated surface.

Passing conformance means the integration honors this contract. It does not certify that an agent is safe, and it is not a regulatory approval.

Versioning

Breaking changes to profile semantics increment the contract major version. Scenario and run-record formats remain backward compatible within a major version; new fields are additive with defaults (for example `execution_mode`, added in v1, defaults to `mediated`).